

Name : Sakshi Himpalnerkar

Batch: B4

class: SY-b(71)

Aim 3. Data Wrangling with pandas: Manipulate and transform data to suit the analysis

THEORY QUESTIONS

1. What is data wrangling? Difference between wrangling and preprocessing?

Data wrangling is the process of cleaning, transforming, and organizing raw data into a structured format suitable for analysis. It includes tasks like handling missing values, removing duplicates, and reshaping data. Data preprocessing, on the other hand, is a broader concept used mainly in machine learning and includes steps like normalization, encoding categorical data, and feature scaling. In simple terms, data wrangling focuses on cleaning and organizing data, while preprocessing prepares the cleaned data for model building.

2. Importance of Exploratory Data Analysis (EDA) in Pandas

Exploratory Data Analysis (EDA) is important because it helps in understanding the dataset before performing analysis or building models. Using Pandas, EDA allows us to summarize data, detect patterns, identify outliers, and find relationships between variables. Functions like `describe()`, `head()`, and `value_counts()` help in gaining insights quickly. EDA ensures that the data is properly understood and improves the quality of analysis.

3. Difference between Series, DataFrame, and Panel

In Pandas, a Series is a one-dimensional array that holds data of a single column. A DataFrame is a two-dimensional table consisting of rows and columns, similar to an Excel sheet, and is the most commonly used structure. A Panel is a three-dimensional data structure used for handling multiple DataFrames, but it is now deprecated in modern Pandas versions. Therefore, Series and DataFrame are widely used in data analysis.

4. Explain the use of functions

a. `isnull()`: This function is used to detect missing values in a dataset. It returns True for missing values and False for non-missing values.

b. `notnull()`: This function is the opposite of `isnull()` and returns True for non-missing values.

c. `dropna()`: This function is used to remove rows or columns that contain missing values.

d. `fillna()`: This function replaces missing values with a specified value such as mean, median, or zero.

5. What is column transformation and why important?

Column transformation is the process of modifying or creating new columns based on existing data. It includes operations like changing data types, applying functions, scaling values, or creating derived columns. It is important because it helps in improving data quality, making the dataset more meaningful, and preparing it for better analysis or machine learning models.

Double-click (or enter) to edit

```
from google.colab import files
uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving employee performance data.csv to employee performance data.csv

```
import pandas as pd
```

```
# Load data
df = pd.read_csv("/content/employee_performance_data.csv")
```

```
# 1. Select single column
print(df['Salary'])
```

```
0      62606.0
1      41534.0
2           NaN
3      31016.0
4     119789.0
5      85591.0
6     119812.0
7      53247.0
8      54300.0
9     104065.0
10     112798.0
11      39268.0
12     116807.0
13     42185.0
14     93704.0
15     116779.0
16     69099.0
17     38571.0
18     68044.0
```

```
19      81214.0
Name: Salary, dtype: float64
```

```
# 2. Add new column with constant value
df['bonus'] = 1000
```

```
# 3. Sort DataFrame by one column
df_sorted = df.sort_values(by='Salary')
print(df_sorted.head())
```

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	\
3	104	David	42	Male	Marketing	2015-12-29	31016.0	
17	118	Rose	23	Female	IT	2005-06-11	38571.0	
11	112	Leo	24	Female	IT	2014-06-24	39268.0	
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0	
13	114	Nina	23	Female	HR	2019-04-23	42185.0	

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week	bonus	
3	7.0		1.0	10977.0	42.0	1000
17	11.0		5.0	12574.0	49.0	1000
11	3.0		3.0	13755.0	41.0	1000
1	1.0		1.0	5206.0	40.0	1000
13	8.0		3.0	17383.0	40.0	1000

```
# 4. Rename columns
df = df.rename(columns={'salary': 'salary_amount'})
```

```
# 5. Summary statistics
print(df.describe())
```

	Employee_ID	Age	Salary	Experience	Performance_Rating
count	20.00000	20.000000	19.000000	19.000000	19.000000
mean	110.50000	40.800000	76338.368421	6.526316	2.789474
std	5.91608	10.909195	31729.124187	4.114587	1.436777
min	101.00000	23.000000	31016.000000	1.000000	1.000000
25%	105.75000	32.000000	47716.000000	3.000000	1.500000
50%	110.50000	42.500000	69099.000000	8.000000	3.000000
75%	115.25000	46.250000	108431.500000	9.500000	4.000000
max	120.00000	59.000000	119812.000000	13.000000	5.000000

	Bonus	Work_Hours_Per_Week	bonus
count	19.000000	19.000000	20.0
mean	11033.421053	39.157895	1000.0
std	3737.670818	5.918550	0.0
min	5206.000000	30.000000	1000.0
25%	8639.000000	35.500000	1000.0
50%	11374.000000	40.000000	1000.0
75%	12647.500000	42.000000	1000.0
max	19820.000000	49.000000	1000.0

```
# 6. Filter rows with multiple conditions
filtered = df[(df['Salary'] > 5000) & (df['Salary'] < 20000)]
```

```
print(filtered)
```

Empty DataFrame

Columns: [Employee_ID, Name, Age, Gender, Department, Join_Date, Salary, Expe
Index: []

```
# 7. Group by column and calculate mean
grouped = df.groupby('Department')['Salary'].mean()
print(grouped)
```

```
Department
HR          83772.833333
IT          70325.666667
Marketing   75119.714286
Name: Salary, dtype: float64
```

```
df2 = pd.DataFrame({
    'Department': ['HR', 'IT'],
    'location': ['Pune', 'Mumbai']
})

merged = pd.merge(df, df2, on='Department', how='inner')
print(merged)
```

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	\
0	103	Charlie	29	Female	IT	2007-02-15	NaN	
1	105	Eve	40	Female	IT	2005-02-04	119789.0	
2	108	Hank	32	Male	IT	2009-05-05	53247.0	
3	109	Ivy	45	Male	IT	2015-10-19	54300.0	
4	111	Kate	45	Female	HR	2007-10-19	112798.0	
5	112	Leo	24	Female	IT	2014-06-24	39268.0	
6	113	Mona	43	Female	HR	2012-02-27	116807.0	
7	114	Nina	23	Female	HR	2019-04-23	42185.0	
8	115	Oscar	45	Female	HR	2015-07-11	93704.0	
9	116	Paul	51	Male	IT	2009-02-09	116779.0	
10	117	Quinn	59	Female	HR	2018-02-20	69099.0	
11	118	Rose	23	Female	IT	2005-06-11	38571.0	
12	119	Steve	42	Male	HR	2016-10-07	68044.0	

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week	bonus	\
0	9.0		4.0	10134.0	33.0	1000
1	9.0		4.0	12721.0	36.0	1000
2	12.0		5.0	12858.0	39.0	1000
3	8.0		NaN	5863.0	42.0	1000
4	11.0		3.0	12408.0	41.0	1000
5	3.0		3.0	13755.0	41.0	1000
6	1.0		1.0	10116.0	49.0	1000
7	8.0		3.0	17383.0	40.0	1000
8	3.0		5.0	NaN	36.0	1000
9	3.0		3.0	6757.0	30.0	1000
10	1.0		1.0	19820.0	30.0	1000
11	11.0		5.0	12574.0	49.0	1000
12	5.0		2.0	11374.0	NaN	1000

```
location
0    Mumbai
1    Mumbai
```

```

2    Mumbai
3    Mumbai
4      Pune
5    Mumbai
6      Pune
7      Pune
8      Pune
9    Mumbai
10     Pune
11    Mumbai
12     Pune

```

```

# 9. Drop rows and columns
df = df.drop(columns=['bonus']) # drop column
df = df.drop(index=0)          # drop row

```

```

# 10. Apply function to column
df['Salary'] = df['Salary'].apply(lambda x: x * 1.1)

print(df.head())

```

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	\
1	102	Bob	36	Female	Marketing	2012-01-01	45687.4	
2	103	Charlie	29	Female	IT	2007-02-15	NaN	
3	104	David	42	Male	Marketing	2015-12-29	34117.6	
4	105	Eve	40	Female	IT	2005-02-04	131767.9	
5	106	Frank	44	Male	Marketing	2013-08-19	94150.1	

	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week
1	1.0		1.0	5206.0
2	9.0		4.0	10134.0
3	7.0		1.0	10977.0
4	9.0		4.0	12721.0
5	NaN		2.0	12035.0